

**INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA**
SUL-RIO-GRANDENSE
Campus Passo Fundo

Bacharelado em Ciência da Computação
Arquitetura de Computadores I

Conteúdo 2:

Aritmética Computacional:

Fundamentos, operações aritméticas, FP (ponto flutuante)

Prof. Petry, Carlos A



Baseado no livro:
Organização e Projeto de Computadores
David Patterson

INTRODUÇÃO: INSTRUÇÕES E CARACTERÍSTICAS

- Computadores usam **palavras** compostas de **conjuntos de bits**
- **Bits** representam **números** (valores) binários
- Números **inteiros** podem ser facilmente **representados em binário**
- Existem **outras representações** numéricas, como podemos armazená-las?
 - Números **não inteiros** e frações
 - Resultados que **excedem limites** de palavras (word)
 - Como o **HW multiplica** ou **divide** números estas representações?
- Este capítulo explora:
 - Representações numéricas
 - Algoritmos aritméticos
 - Implicações no desempenho aritmético de programas

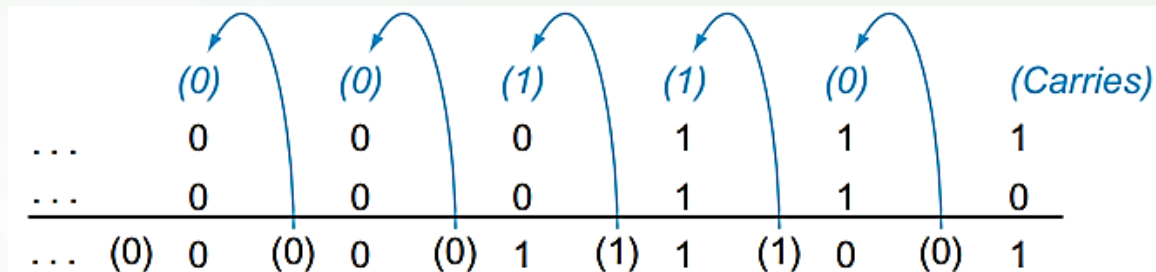
ADIÇÃO E SUBTRAÇÃO

- Adição

- Dígitos binários (bits) são somados **como em decimal**, da **direita para esquerda**, levando **'vai-um'** ao próximo dígito, exemplo:

$$\begin{array}{r} 00000000 \ 00000000 \ 00000000 \ 00000111_2 = 7_{10} \\ + \ 00000000 \ 00000000 \ 00000000 \ 00000110_2 = 6_{10} \\ \hline 00000000 \ 00000000 \ 00000000 \ 00001101_2 = 13_{10} \end{array}$$

- A dinâmica da adição pode ser assim descrita, 'vai-um' entre parênteses:



ADIÇÃO E SUBTRAÇÃO

- Subtração

- Dígitos binários (bits) são subtraídos **como em decimal**, da **direita para esquerda**, levando 'vai-um' ao próximo dígito, exemplo:

$$\begin{array}{r} 00000000 \ 00000000 \ 00000000 \ 00000111_2 = 7_{10} \\ - 00000000 \ 00000000 \ 00000000 \ 00000110_2 = 6_{10} \\ \hline 00000000 \ 00000000 \ 00000000 \ 00000001_2 = 1_{10} \end{array}$$

- Em **complemento de dois** a subtração fica assim apresentada:

$$\begin{array}{r} 00000000 \ 00000000 \ 00000000 \ 00000111_2 = 7_{10} \\ + 11111111 \ 11111111 \ 11111111 \ 11111010_2 = -6_{10} \\ \hline 00000000 \ 00000000 \ 00000000 \ 00000001_2 = 1_{10} \end{array}$$

ADIÇÃO E SUBTRAÇÃO

- Overflow (estouro de capacidade) em operações aritméticas
 - Ocorre quando o **resultado** da operação **excede o número de bits** da palavra (ex: 32 bits)
 - Na adição **não ocorre overflow** ao somar **operandos com sinais diferentes**
 - Devido à soma não ser maior que um dos operandos, exemplo: $-10 + 4 = -6$
 - Como os dois operandos cabem na palavra, a resposta também caberá
 - Na subtração **não ocorre overflow** ao diminuir **operandos com sinais iguais**
 - Uma vez que $(+) x - y = x + (-y)$, portanto uma soma com sinais diferentes

ADIÇÃO E SUBTRAÇÃO

- Quando ocorre overflow
 - Na adição: ao somar dois números positivos cujo resultado gere o 33° bit
 - Na subtração:
 - (1) ao diminuir um número negativo de um positivo e resultar um negativo,
 - (2) ao diminuir um número positivo de um negativo e resultar um positivo
- Resumo das condições onde ocorrem overflow na adição e subtração

Operação	Operando A	Operando B	Resultado indicando overflow
A+B	≥ 0	≥ 0	< 0
A+B	< 0	< 0	≥ 0
A-B	≥ 0	< 0	< 0
A-B	< 0	≥ 0	≥ 0

ADIÇÃO E SUBTRAÇÃO

- Overflow em operações aritméticas com e sem sinal
 - Adição e subtração **com sinal** precisam **tratar** a ocorrência de **overflow**
 - Operações **sem sinal** costumam referenciar endereços de memória, neste caso o **overflow** pode ser **ignorado**
- MIPS implementa instruções para atender à duas condições
 - Instruções com sinal `add`, `addi` e `sub` **tratam overflow** via exceções
 - Instruções sem sinal `addu`, `addiu` e `subu` **não tratam overflow**, ignoram
- Exceção
 - Tratada via interrupção: evento que interrompe o processamento e desvia para uma rotina de tratamento de overflow

MULTIPLICAÇÃO E DIVISÃO

- Operações de adição e subtração possuem um algoritmo de solução mais simples
- Operações de **multiplicação** e **divisão** empregam algoritmos de solução **mais complexos**
- Assim, operações de adição e subtração terão hardware mais simples e executarão mais rapidamente (menos ciclos de clock)
- Enquanto, operações de multiplicação e divisão exigirão hardware mais complexo e executarão mais lentamente (mais ciclos de clock)

MULTIPLICAÇÃO

- Por serem mais complexas, **multiplicações** exige um hardware **mais elaborado** que adição e subtração
- Método da **multiplicação** decimal:
 - Por simplicidade usamos apenas dígitos 0 e 1
 - Binários operam de forma semelhante
- O **produto** possuirá **mais dígitos** do que os operandos
- Sendo m e n a largura em bits dos operandos, temos:
 $m+n$ como a largura em bits do produto, ignorando o bit de sinal
- Assim, a multiplicação também pode produzir overflow, exigindo trata-lo via exceção

Multiplicando	1000
Multiplicador	$\times 1001$

	1000
	0000
	0000
	1000

Produto	1001000 ₁₀

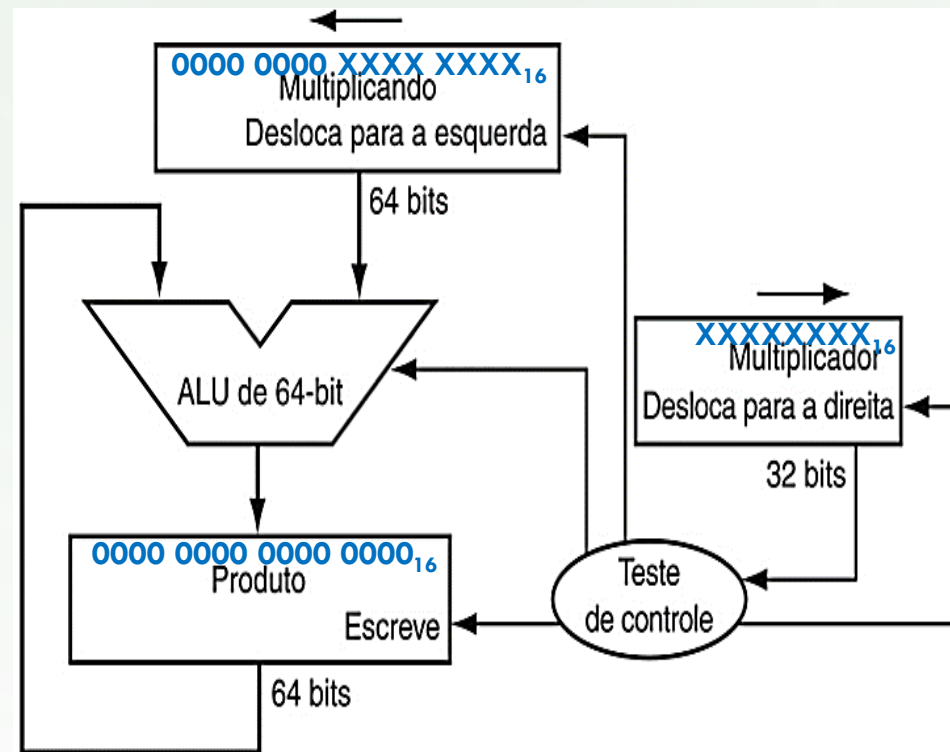
MULTIPLICAÇÃO:

DIAGRAMA DO HARDWARE DE MULTIPLICAÇÃO: VERSÃO BÁSICA (PRIMÁRIA)

- Componentes:

- Multiplicando (**Mdo**)¹, Produto (**Pto**)¹ e ALU 64b, Multiplicador(**Mor**)¹ 32b
- Teste: unidade de controle da multiplicação
- Conteúdo: 0 → zero, X → número (hexadecimal)

- Diagrama do Hardware



¹ Registradores

MULTIPLICAÇÃO:

ALGORITMO DO HARDWARE DE MULTIPLICAÇÃO: VERSÃO BÁSICA (PRIMÁRIA)

- Operações

0. Início, carrega valores nos registradores:

$Mdo=num$, $Mor=num$, $Pto=00$

1. Testa o LSD de Mor :

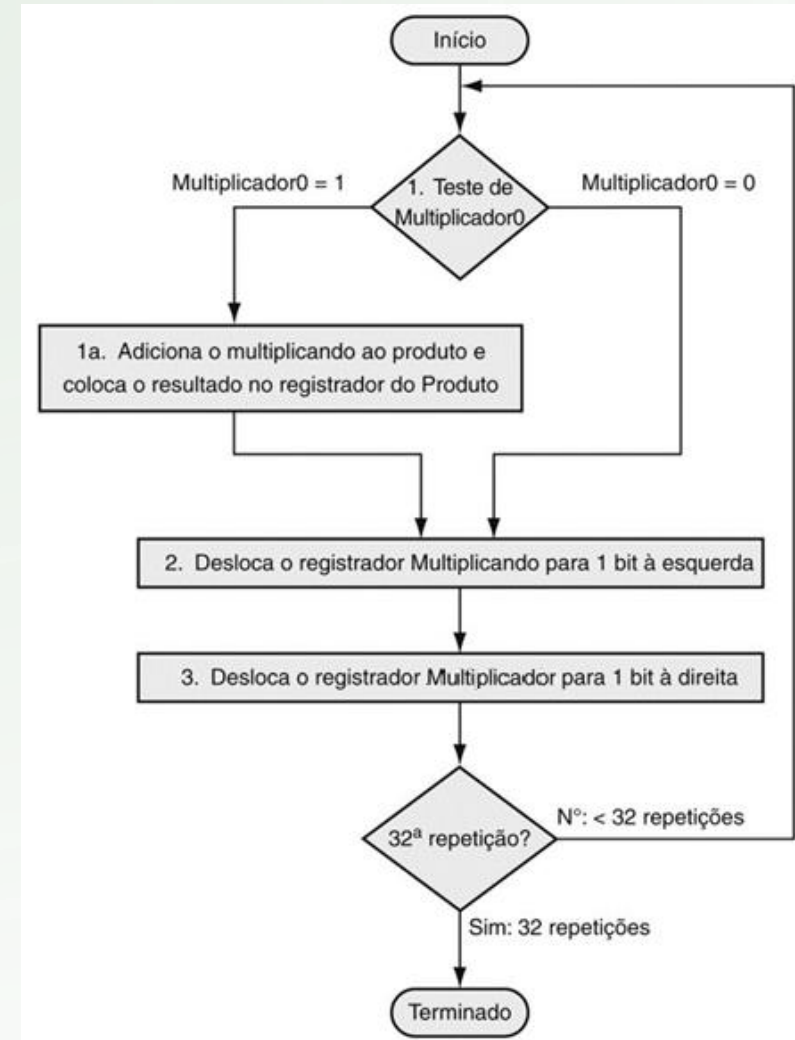
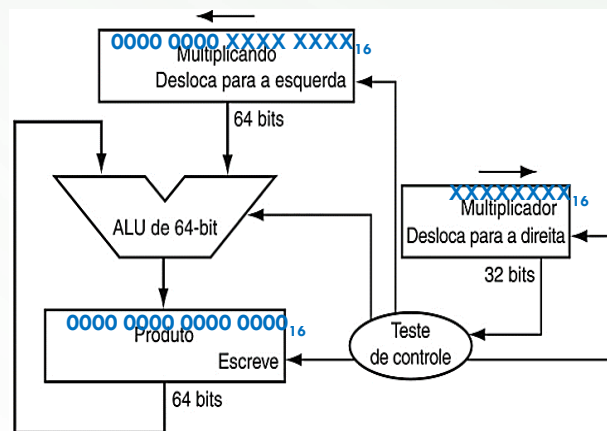
$LSD(Mor) ? 1 \rightarrow Pto += Mdo : 0 \rightarrow Pto = Pto$

2. Let $Mdo = Mdo \ll 1$

3. Let $Mor = Mor \gg 1$

4. Término, testa o n° de iterações:

$(Iter < 32) ? \rightarrow Volta_1 : \rightarrow Termina$

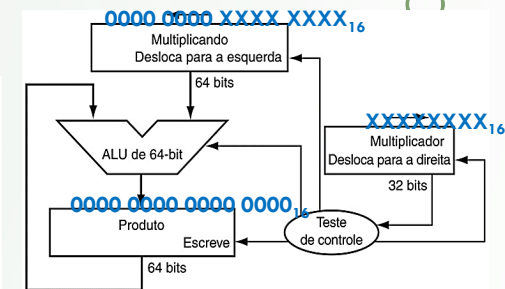


MULTIPLICAÇÃO:

ITERAÇÃO DA MULTIPLICAÇÃO EM HARDWARE: VERSÃO BÁSICA (PRIMÁRIA)

- Exemplo: sequência de multiplicação de $2 * 3$ ou 0010_2 (Mdo) $\times$ 0011_2 (Mor)
 - Por simplificação, usamos apenas 4 dígitos (bits)

Iteração	Passo	Reg Mdo*	Reg Mor*	Reg Pto*
0	Carrega valores iniciais	0000 0010	0011	0000 0000
1ª	1. LSD(Mor) ? Pto+=Mdo :	0000 0010	001 1	0000 0010
	2. Let Mdo=Mdo<<1	0000 0100	0011	0000 0010
	3. Let Mor=Mor>>1	0000 0100	0001	0000 0010
2ª	1. LSD(Mor) ? Pto+=Mdo :	0000 0100	000 1	0000 0110
	2. Let Mdo=Mdo<<1	0000 1000	0001	0000 0110
	3. Let Mor=Mor>>1	0000 1000	0000	0000 0110
3ª	1. LSD(Mor) ? :Pto=Pto	0000 1000	000 0	0000 0110
	2. Let Mdo=Mdo<<1	0001 0000	0000	0000 0110
	3. Let Mor=Mor>>1	0001 0000	0000	0000 0110
4ª	1. LSD(Mor) ? :Pto=Pto	0001 0000	000 0	0000 0110
	2. Let Mdo=Mdo<<1	0010 0000	0000	0000 0110
	3. Let Mor=Mor>>1	0010 0000	0000	<u>0000 0110</u>



* (1) 4 bits: tamanho da palavra (2) **Negrito**: mudança de valores em registradores (3) **Azul**: bit (LSD) testado no registrador multiplicador (Mor)

MULTIPLICAÇÃO:

MULTIPLICAÇÃO POR DESLOCAMENTO EM HARDWARE

- **Compiladores** podem **alterar o código** para aumentar o desempenho
- Por exemplo, substituir a multiplicação aritmética por deslocamento lógico
- Deslocamento lógico:
 - **Deslocamento** lógico para **esquerda** equivale a multiplicar por **potência de 2**
 - Exemplo: $7 * 4 = 28$, $7 \ll 2 = 7 * 2^2 = 28$
 - A **multiplicação** pode ser **substituída por deslocamento** e eventuais somas
 - Exemplo: $7 * 5 = 35 \rightarrow 7 * 2^2 + 7 = 7 \ll 2 + 7 = 35$
- Outras otimizações para aumentar desempenho da multiplicação
 - Empregar a multiplicação em paralelo dos bits, usando árvores de somadores
 - Empregar pipeline: divisão da multiplicação em etapas executando-as em paralelo

DIVISÃO

- A **divisão** também exige hardware mais **complexo**, comparado ao da **multiplicação**
 - Ademais, a divisão precisa tratar resultados inválidos: **divisão por 0**
- Método da **divisão decimal**:
 - Por simplicidade usamos apenas dígitos 0 e 1
 - Exemplo: $1.001.010 / 1000$
 - Binários operam de forma semelhante
- A divisão possui 4 elementos
 - Dividendo, Divisor, Quociente e Resto
- Pode ser expressa como:
 - $\text{Dividendo} = \text{Quociente} \times \text{Divisor} + \text{Resto}$
 - $\text{Dividendo} / \text{Divisor} = \text{Quociente} + \text{Resto}$

			1001	Quociente ₁₀
Divisor	1000		1001010	Dividendo
			-1000	
			10	
			101	
			1010	
			-1000	
			10	Resto ₁₀

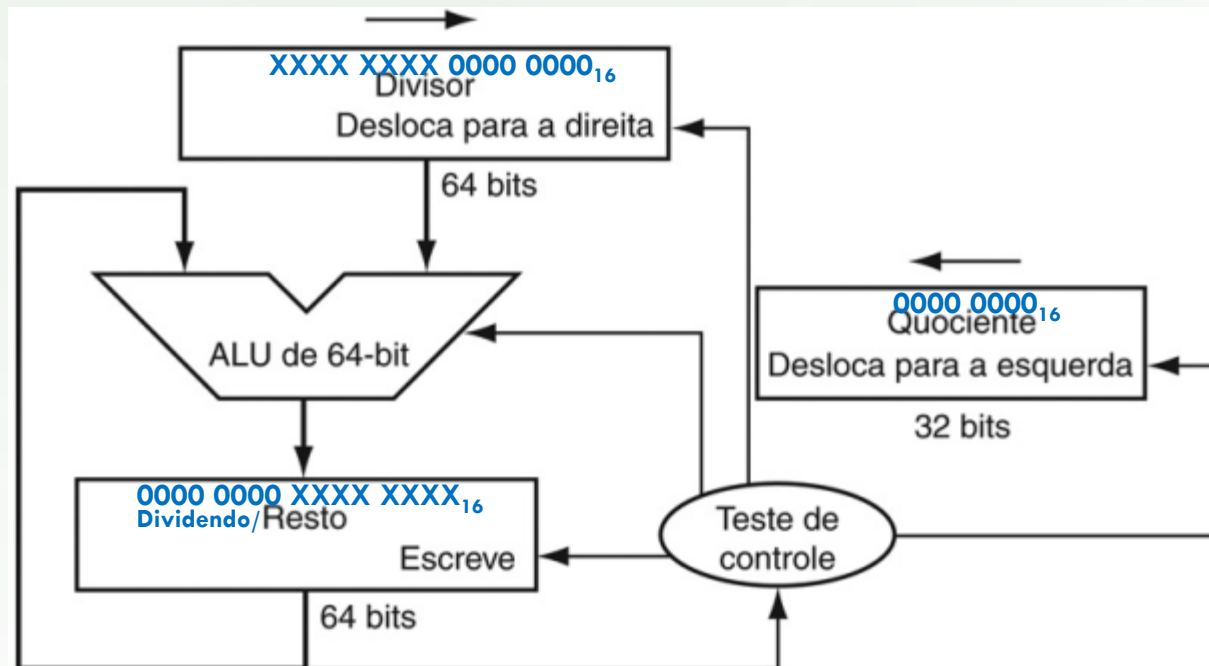
DIVISÃO:

DIAGRAMA DO HARDWARE DE DIVISÃO: VERSÃO BÁSICA (PRIMÁRIA)

- Componentes:

- Dividendo/Resto(**Ddo**/**Rto**)¹, Divisor(**Dor**)¹ e ALU 64b, Quociente (**Qte**)¹ 32b
- Teste: unidade de controle da divisão
- Conteúdo: **0** → zero, **X** → número (hexadecimal)

- Diagrama do Hardware



DIVISÃO:

ALGORITMO DO HARDWARE DE DIVISÃO: VERSÃO BÁSICA (PRIMÁRIA)

• Operações

0. Início, carrega valores nos registradores:

$$Rto^1 = Ddo, Dor = UP(num)^2, Qte = 0$$

1. Let $Rto \leftarrow Dor^3$; Let $Qte = Qte \ll 1$

2. Testa o registrador Rto^4 :

$$MSD(Rto) == 0 ? SetLSD(Qte) : Let Rto += Dor; ResetLSD(Qte)$$

3. Let $Dor = Dor \gg 1$

4. Término, testa o n° de iterações:

$$(Iter < 33) ? \rightarrow Volta_1 : \rightarrow Termina$$

¹ Registrador **Resto**:

- Inicia com o valor do Dividendo
- Mantém o Dividendo ao longo da divisão
- Conterá o Resto ao finalizar a divisão

² Registrador **Divisor**:

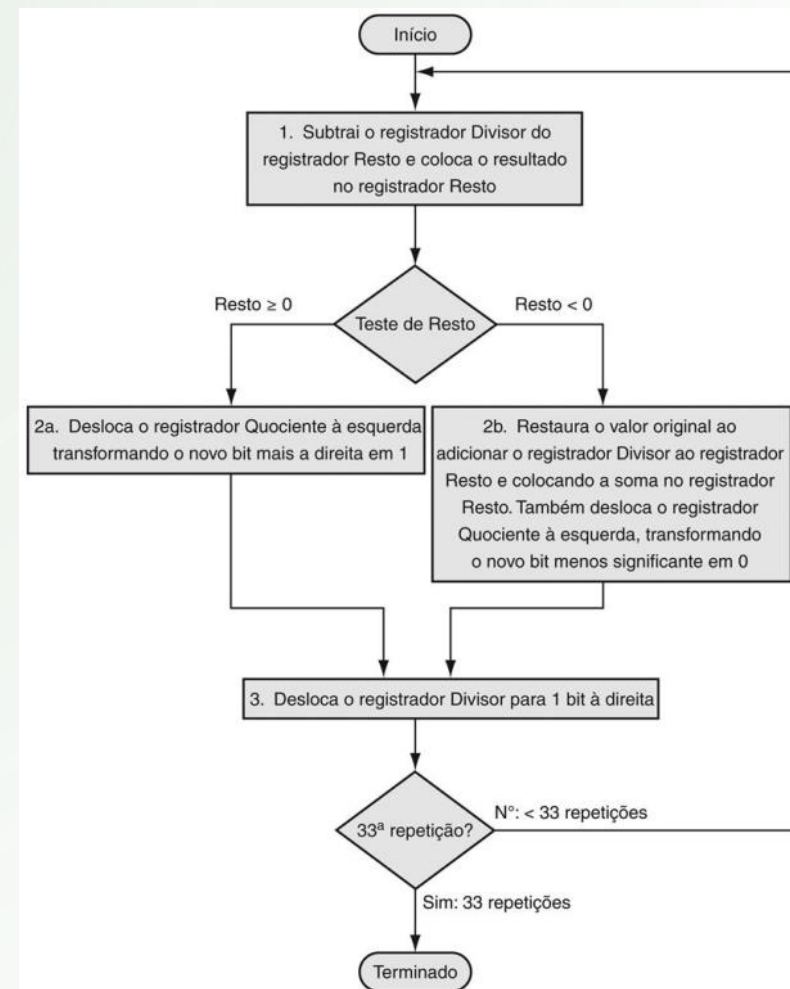
- UpPart_Dor = num

³ Subtração Resto-Divisor:

- Positivo : Divisor $\leq$ Dividendo
- Negativo: Divisor $>$ Dividendo

⁴ Teste Resto/Dividendo:

- Desloca Divisor até caber no Dividendo
- Então: Set LSD do Quociente senão Reset LSD



DIVISÃO:

ITERAÇÃO DA DIVISÃO EM HARDWARE: VERSÃO BÁSICA (PRIMÁRIA)

- Exemplo: sequência de divisão de $7/2$ ou 0111_2 (Ddo)/ 0010_2 (Dor), ambos 4 bits

Iteração	Passo	Reg Rto/Ddo *	Reg Dor *	Reg Qte *
0	Carrega valores iniciais	0000 0111	<u>0010</u> 0000	0000
1ª	1. Let Rto -= Dor , Qte <<1 2. (MSD(Rto)=0) ? : Let Rto += Dor , ResetLSD(Qte) 3. Let Dor = Dor >>1	1 110 0111 0000 0111 0000 0111	0010 0000 0010 0000 0001 0000	0000 0000 0000
2ª	1. Let Rto -= Dor , Qte <<1 2. (MSD(Rto)=0) ? : Let Rto += Dor , ResetLSD(Qte) 3. Let Dor = Dor >>1	1 111 0111 0000 0111 0000 0111	0001 0000 0001 0000 0000 1000	0000 0000 0000
3ª	1. Let Rto -= Dor , Qte <<1 2. (MSD(Rto)=0) ? : Let Rto += Dor , ResetLSD(Qte) 3. Let Dor = Dor >>1	1 111 1111 0000 0111 0000 0111	0000 1000 0000 1000 0000 0100	0000 0000 0000
4ª	1. Let Rto -= Dor , Qte <<1 2. (MSD(Rto)=0) ? SetLSD(Qte) : 3. Let Dor = Dor >>1	0 000 0011 0000 0011 0000 0011	0000 0100 0000 0100 0000 0010	0000 0001 0001
5ª	1. Let Rto -= Dor , Qte <<1 2. (MSD(Rto)=0) ? SetLSD(Qte) : 3. Let Dor = Dor >>1	0 000 0001 0000 0001 <u>0000</u> 0001	0000 0010 0000 0010 0000 0001	0010 0011 <u>0011</u>

* (1)4 bits: tamanho da palavra (2)**Negrito**: mudança de valores em registradores (3)**Azul**: bit (MSD) testado no registrador Resto (Rto)

MIPS ASSEMBLY:

RESUMO DE INSTRUÇÕES DE PROCESSADOR E GRUPOS

Linguagem de assembly MIPS

Categoria	Instrução	Exemplo	Significado	Comentários
Aritmética	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Três operandos, overflow detectado
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Três operandos, overflow detectado
	add immediate	addi \$s1,\$s2,100	$\$s1 = \$s2 + 100$	+ constante, overflow detectado
	add unsigned	addu \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$	Três operandos, overflow detectado
	subtract unsigned	subu \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$	Três operandos, overflow detectado
	add immediate unsigned	addiu \$s1,\$s2,100	$\$s1 = \$s2 + 100$	+ constante, overflow detectado
	move from coprocessor register	mfc0 \$s1,\$epc	$\$s1 = \epc	Copiar Exceção PC + regs especiais
	multiply	mult \$s2,\$s3	Hi, Lo = $\$s2 \times \$s3$	Produto de 64-bit com sinal em Hi, Lo
	multiply unsigned	multu \$s2,\$s3	Hi, Lo = $\$s2 \times \$s3$	Produto de 64-bit sem sinal em Hi, Lo
	divide	div \$s2,\$s3	Lo = $\$s2 / \$s3$, Hi = $\$s2 \bmod \$s3$	Lo = quociente, Hi = resto
	divide unsigned	divu \$s2,\$s3	Lo = $\$s2 / \$s3$, Hi = $\$s2 \bmod \$s3$	Quociente sem sinal e resto
	move from Hi	mghi \$s1	$\$s1 = \text{Hi}$	Costumava copiar de Hi
	move from Lo	mflo \$s1	$\$s1 = \text{Lo}$	Costumava copiar de Lo
Transferência de dados	load word	lw \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Word da memória para o registrador
	store word	sw \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Word do registrador para a memória
	load half unsigned	lhu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Halfword da memória para o registrador
	store half	sh \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Halfword do registrador para a memória
	load byte unsigned	lbu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$	Byte da memória para o registrador
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$	Byte do registrador para a memória
	load upper immediate	lui \$s1,100	$\$s1 = 100 \times 2^{16}$	Carrega constante nos 16 bits altos
Lógica	AND	AND \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$	Três operandos registradores; AND bit por bit
	OR	OR \$s1,\$s2,\$s3	$\$s1 = \$s2 \mid \$s3$	Três operandos registradores; OR bit por bit
	NOR	NOR \$s1,\$s2,\$s3	$\$s1 = \sim (\$s2 \mid \$s3)$	Três operandos registradores; NOR bit por bit
	AND immediate	ANDi \$s1,\$s2,100	$\$s1 = \$s2 \& 100$	AND bit por bit com constante
	OR immediate	ORi \$s1,\$s2,100	$\$s1 = \$s2 \mid 100$	OR bit por bit com constante
	shift left logical	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$	Desloca à esquerda pela constante
	shift right logical	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$	Desloca à direita pela constante
Desvio condicional	branch on equal	beq \$s1,\$s2,25	if ($\$s1 == \$s2$) go to PC + 4 + 100	Teste idêntico; desvio relativo ao PC
	branch on not equal	bne \$s1,\$s2,25	if ($\$s1 \neq \$s2$) go to PC + 4 + 100	Teste não-idêntico; relativo ao PC
	set on less than	slt \$s1,\$s2,\$s3	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	Comparação menos que; complementos de dois
	set less than immediate	slti \$s1,\$s2,100	if ($\$s2 < 100$) $\$s1 = 1$; else $\$s1 = 0$	Comparação < constante; complementos de dois
	set less than unsigned	sltu \$s1,\$s2,\$s3	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$	Comparação menos que; números naturais
	set less than immediate unsigned	sltiu \$s1,\$s2,100	if ($\$s2 < 100$) $\$s1 = 1$; else $\$s1 = 0$	Comparação < constante; números naturais
Pulo incondicional	jump	j 2500	go to 10000	Pula para o endereço alvo
	jump register	jr \$ra	go to \$ra	Para troca, procedimento de retorno
	jump and link	jal 2500	$\$ra = \text{PC} + 4$; go to 10000	Para chamada de procedimento

FLOATING POINT – FP:

ARMAZENAMENTO EM NOTAÇÃO DE PONTO FLUTUANTE

- Ponto flutuante (FP) possibilita um método para armazenar valores não inteiros
- Valores fracionários, com parte decimal ou em notação científica, por exemplo:
 - 3.1415926 (π) ou 2.718281 (e)
 - $0.000000001 = 1.0 \times 10^{-9}$ (nanosegundo)
 - $3,155,760,000 = 3.15576 \times 10^9$ (quantidade de segundos em um século)
- Representar valores em FP implica usar a base 2 ao invés da base 10
- Representações em FP são armazenadas usando múltiplos de palavras (ex: 32b, 64b)
- Vantagens do uso de FP
 - Simplifica a troca de dados entre sistemas computacionais
 - Simplifica algoritmos de manipulação de valores reais
 - Aumenta a precisão numérica de valores armazenados

REPRESENTAÇÃO EM FP:

- O formato FP possui o seguinte formato **normalizado**:
 - $1.(xxxxxxxx)_{\text{bin}} \times 2^{yyyy}$ (fração $\times$ potência de base 2)
- A representação em **FP** possui **duas partes**:
 - **Fração** : valor representando a parte não inteira (entre 0 e 1)
 - **Expoente** : valor representando uma potência de base 2
- Exemplo de representação de número real em **FP**:
 - $13.25_{\text{dec}} \Rightarrow 1101.01_{\text{bin}}$
 - Normalizando $\Rightarrow 1.\underline{10101}_{\text{bin}} \times 2^3$
- Em geral, como no MIPS, a **representação** de **FP** é composta por **3 campos**:
 1. **Sinal** do número (como um todo)
 2. Valor do **expoente** (armazenado como **natural**, porém usa o mecanismo de **bias**)
 3. **Fração** (valor contendo a representação não inteira)

REPRESENTAÇÃO EM FP:

• Exemplo de FP 32b normalizado $= 13.25_{\text{dec}} \Rightarrow 1101.01_{\text{bin}} \Rightarrow \boxed{1.\underline{10101}_{\text{bin}} \times 2^3}$

• **Sinal** (1b): 0 pois (todo) o número é positivo

• **Expoente** (8b): 0000 0011 (3_{dec}) mas tratado no formato **bias** (excesso)

• Valor bias: $2^{d-1}-1$, onde d = nº bits do expoente

• Ex: bias de 8b $\Rightarrow 2^{8-1}-1 = 127$ - bias de 11b $\Rightarrow 2^{11-1}-1 = 1023$

• Armazenando o expoente: $\text{exp} + \text{bias}$

Ex: $2^3 \Rightarrow \text{Exp: } 3$ (00000011) $\Rightarrow 3 + 127 \Rightarrow 130$ (10000010)

• Lendo o expoente: $\text{exp} - \text{bias}$

Ex: 130 (10000010) $\Rightarrow 130 - 127 \Rightarrow \text{Exp } 3$ (00000011) $\Rightarrow 2^3$

• **Fração** (23b): $1.\underline{10101}$ (13.25_{dec})

• O bit inteiro (1 mais à esquerda) NÃO é armazenado (bit implícito, pois SEMPRE será 1)

• Armazena a mantissa decimal (não inteiro): $\underline{101010} \dots 0$ (0s até preencher os 23 bits)

REPRESENTAÇÃO EM FP: FORMATO PRECISÃO SIMPLES

- A **representação** composta por 3 campos é denominada: **sinal-magnitude**
- Para uma palavra de **32 bits** temos a seguinte **distribuição de campos**:



- A representação matemática dos campos é dada por:

$$(-1_2)^s \times F \times 2^E$$

- s : representa o sinal do número
- F : representa a própria fração (considerar o 1 implícito)
- E : representa o expoente do número (usa o formato bias)
- Faixa de valores representáveis $\cong 2.0 \times 10^{-38}$ a $2.0 \times 10^{+38}$

REPRESENTAÇÃO EM FP:

- O mesmo processo é aplicado se o expoente fosse negativo, por exemplo: 2^{-4}
- O valor 13.25_{dec} em FP 32b normalizado é armazenado como:

Sinal (1b)	Exp (8b)	Mantissa (23b)
0	1000 0010	101 0100 0000 0000 0000 0000

- Para se obter o equivalente decimal usamos a fórmula: $(-1_2)^S \times F \times 2^E$

$$(-1_2)^S \times F \times 2^E$$

$$(-1)^0 \times 1.10101 \times 2^{(1000\ 0010)-127} =$$

$$(-1_2)^0 \times 1.10101 \times 2^{(130)-127} =$$

$$1 \times 1.10101 \times 2^3 =$$

$$1 \times 1101.01 =$$

$$1 \times (1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0) + (0 \times 2^{-1} + 1 \times 2^{-2}) =$$

$$1 \times (8 + 4 + 0 + 1) + (0 + 0.25) =$$

$$1 \times 13 + 0.25 =$$

$$1 \times 13.25 =$$

$$13.25$$

REPRESENTAÇÃO EM FP: FORMATO PRECISÃO DUPLA

- Em FP também pode ocorrer **overflow**:
 - Quando um **expoente positivo** torna-se **grande demais** para caber no campo de expoente
- Em FP também pode ocorrer **underflow**:
 - Quando um **expoente negativo** torna-se **grande demais** para caber no campo de expoente
- Para **reduzir (over | under)flow**, podemos usar a precisão dupla
 - Usamos **duas palavras** para armazenar números em FP, por exemplo: 64 bits
 - Ampliamos o número de bits dos campos: fração e expoente
 - Faixa de valores representáveis $\cong 2.0 \times 10^{-308}$ a $2.0 \times 10^{+308}$



REPRESENTAÇÃO EM FP: FORMATO IEEE754

- A norma **IEEE754** amplia a faixa de representação de **valores em FP**
 - O IEEE 754 torna **implícito** o **bit 1 inicial** dos números binários normalizados
 - Portanto, não precisa ser incluído ao armazenar o número
 - Efeito: a fração passa a representar **24 bits** ao invés de 23 bits e **53 bits** em vez de 52 (em precisão dupla)
 - Usando **IEEE754** a **fração** passa a ser chada de **significando**
- Assim, temos dois **termos** para indicar a **fração**:
 - **Significando** indicando $\rightarrow 1_2 + \text{fração}$
 - **Fração** indicando $\rightarrow$ a própria fração
- A representação matemática em IEEE754 :
 - Define valores representados pela fórmula¹: $(-1_2)^s \times (1_2 + F) \times 2^E$
 - Possui valores reservados: como o zero (0) $\rightarrow$ para o valor 0.0×0 não adicionado o 1

¹ s: sinal 0 | 1 - + indica concatenação

REPRESENTAÇÃO EM FP: FORMATO IEEE754

- A representações de **valores em FP**, usando o padrão de codificação **IEEE754**, é resumida na tabela abaixo:

Precisão Simples		Precisão Dupla		Representação
Expoente	Fração	Expoente	Fração	-
0	0	0	0	0
0	não zero	0	não zero	$\pm$ num não-normalizado
1-254	qualquer coisa	1-2046	qualquer coisa	$\pm$ num FP normalizado
255	0	2047	0	$\pm$ valor infinito
255	não zero	2047	não zero	Not a Number (NaN)

- NaN: representa resultados inválidos, como divisão por 0, subtração entre infinitos, possibilitando avaliação posterior à exceção
- O sinal separado facilita comparações, especialmente para ordenações
- O expoente antes do significando facilita operações de ordenação de FP

BACKUP

REPRESENTAÇÃO EM FP: EXPOENTES EM IEEE754

- Usando **complemento de 2** para representar o **expoente**:
 - Leva a expoentes **negativos** parecerem **maiores do que** os **positivos**
 - Devido ao **MSD** possuir **1** em expoentes **negativos**
- **Exemplo**: representando **expoentes + e -** em complemento de 2:

- $1,0_2 \times 2^{-1}$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

- $1,0_2 \times 2^{+1}$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.

- Lembre: **1** está implícito como MSD no **significando** (**1**.bbb...)
- Convertendo os bits 30-23 (**expoente**) em **decimal** equivalente, teríamos:
 $2^{+1} \rightarrow 1$ e $2^{-1} \rightarrow 255$, mesmo em binário o valor do expoente negativo é maior

REPRESENTAÇÃO EM FP: EXPOENTES EM IEEE754

- Alternativa: **notação deslocada** para representar o **expoente**
 - Representar o **menor expoente** negativo como $0...0_2$
 - Representar o **maior expoente** positivo como $1...1_2$
 - IEEE754 usa o **bias** (divisor) **127** (em precisão simples) para definir **expoentes** + e –
 - Exemplos
 - 2^{-1} é representado como $-1+127$ ou 126 ou 0111 1110, 2^{-2} como 125 ... 2^{-126} como 1
 - 2^{+1} é representado como $+1+127$ ou 128 ou 1000 0000, 2^{+2} como 129 ... 2^{+127} como 254
 - A representação em **FP** usando **notação deslocada** torna-se:
$$(-1_2)^S \times (1_2 + F) \times 2^{(E - \text{Bias})}$$

Exemplo: $2^{126} (0111\ 1110) = 2^{+126-127} = 2^{-1}$ e $2^{128} (1000\ 0000) = 2^{128-127} = 2^1$
 - A faixa de valores em precisão simples (23+1 bits) será:
Menor $n^\circ \pm 1.000000000000000000000000 \times 2^{-126} (\pm 1.17_{10}^{-38})$
Maior $n^\circ \pm 1.111111111111111111111111 \times 2^{+127} (\pm 3.4_{10}^{+38})$
 - Zero representado como 0.000000000000000000000000 e0
 - Infinito representado como $\pm 000000000000000000000000$ e255

REPRESENTAÇÃO EM FP: EXPOENTES EM IEEE754

- Exemplo 1: representar $-0,75_{10}$ como binário IEEE754 em precisão simples

- O valor $-0,75$ também pode ser representado como: $-3/4$ ou $-3/2^2$
- Em binário corresponde à fração: $-11_2/2^2$ ou -0.11_2
- Em notação científica representamos como: $-0.11_2 \times 2^0$
- Em notação científica normalizada ficará: $-1.1_2 \times 2^{-1}$
- Usando o formato IEEE754 $(-1_2)^s \times (1_2 + F) \times 2^{(E - \text{Bias})}$ ficará:
 - Sinal: 1, indicando valor negativo
 - Expoente: valor 126 (0111 1110) representado o expoente -1 ($126 - 127$)
 - Fração: -1.1 Significando: $1 + .1000...$

$$(-1_2)^1 \times (1 + .100000000000000000000000000000) \times 2^{(126 - 127)}$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	1	1	1	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
1 bit									23 bits																						
8 bits																															

REPRESENTAÇÃO EM FP: EXPOENTES EM IEEE754

- Exemplo 2: considere o valor IEEE754 em precisão simples a seguir

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

- A representação equivalente em decimal será a seguinte:
 - Usando o formato IEEE754 $(-1_2)^S \times (1_2 + F) \times 2^{(E - \text{Bias})}$ ficará:
 - Sinal: 1, indicando valor negativo
 - Expoente: valor 129 (1000 0001) representado o expoente 2 (129-127)
 - Significando: 1+.0100... Fração: 1+0.25
 - Substituindo valores no formato, teremos:

$$(-1_2)^1 \times (1+.0100000000000000000000000000) \times 2^{(126-127)}$$

$$-1 \times 1.25 \times 2^2 = -1.25 \times 4 = -5.0$$